



Universidad Nacional Experimental de Guayana
Vicerrectorado Académico
Proyecto de Carrera: Ingeniería en Informática
Unidad Curricular: Lenguajes y Compiladores

Autómatas de Pila

Lexer Docker

Lenguaje Rust

FLEX

ANÁLISIS LÉXICO

Los ASTronautas

“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:
Félix Márquez

Presentado por:

C.I.: V-25.933.680. Albuquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis

Autómatas de Pila: Definición

ACTIVIDAD 1 — TEORÍA DE AUTÓMATAS

Un autómata de pila es un modelo matemático que extiende los autómatas finitos añadiendo una memoria auxiliar llamada PILA.

LA PILA (MEMORIA LIFO)

Último en entrar → Primero en salir

DEFINICIÓN FORMAL (7-tupla):

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Q = Estados

Σ = Alfabeto de entrada

Γ = Alfabeto de pila

δ = Función de transición

q_0 = Estado inicial

Z_0 = Símbolo inicial de pila

F = Estados de aceptación

$\delta(q, a, X) = (\text{nuevo_estado}, \text{modificación_pila})$

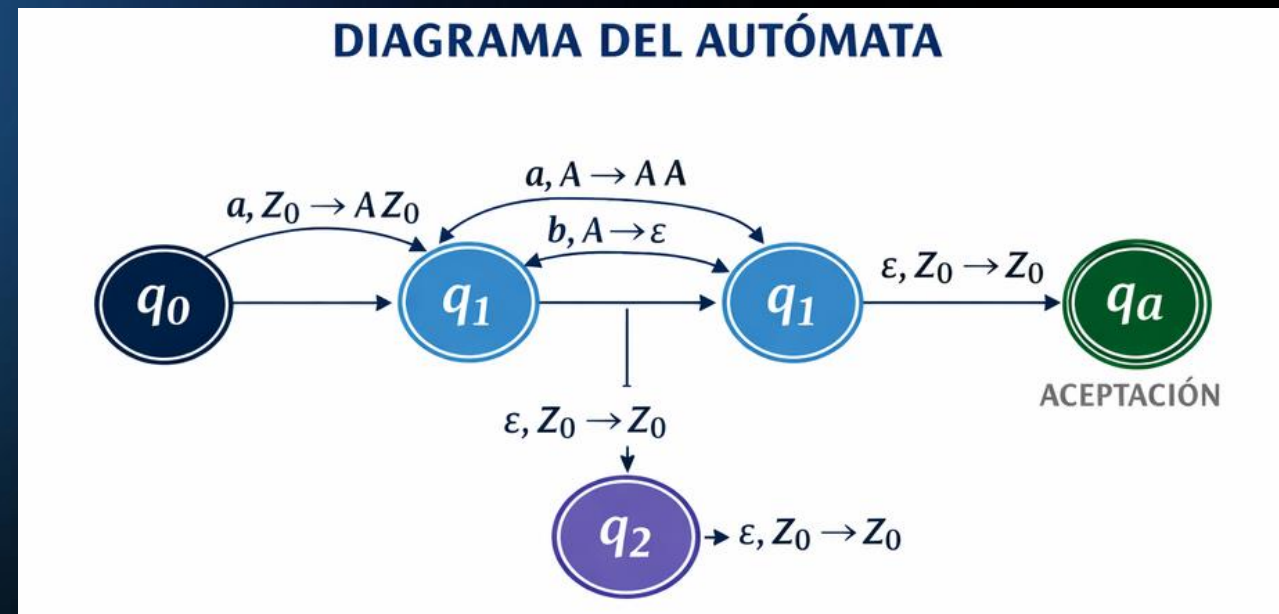
```
14 pf_natiime {urs_aitomatosiph=>20};
6   spitlee1_sti=non*);_lcohes#1 = (#5.1))
12
12 difebsturm uar=muineta(K)|;
3   nitlequaity.1(la4vidas()
2   rEoppes_donal_kriac_satiicns==Poaraite 220;
9
17   -fuálikamess()
30   /s=-(leapac ádiop=, frete-bactor™);)
72   rrou duaf), +Zlop, #1, 'cátweét iony.5'ac!)
33   1.<f*-(Blscime1",-eC), tienstleat; 13(≡ lop,,
54   .altenae(')|Y), ómantai";->fúc(nacáioa5));
00   7 m(dád bríiaed,('))>Drañier,'cuver"am.
87
35   mf=-(candell); autonnaticLiesc#]
24   tnstiliitor suatercus, hac'e88";
22   trafilaiger/spitict: '"dupler:-inps4)
34   trancutjec omt9land-'alif,
28
```

Ejemplos de Autómatas de Pila

ACTIVIDAD 1 — AUTÓMATAS DE PILA

EJEMPLO: LENGUAJE $a^n b^n$ Lenguaje: $\{ a^n b^n \mid n \geq 1 \}$

Ejemplo: "aaabbb"



SIMULACIÓN DE "aaabbb":

a → apila A

a → apila A

a → apila A

b → desapila A

b → desapila A

b → desapila A

RESULTADO: ¡CADENA ACEPTADA!

```
ptackk 0standetatec;, ()
ansip: --aNS
pushh: ( kt.isapp):
prash: (suf ted_hyhin
Pushh: (aodabit Ehtatte;
      =  -+k /
pushh: ( k.f.?.exp,
```

```
ptackk starcvaneipr;, ()
pushh: gw:ipk;
pushh: (e ia.t.sem pla;
      =  asq;'> [-]=e
pushh: (_tlhe NntiHuna, a trenn=tbYIN+)u+;)
pushh: (ci.ep-2 ifaprosest:τ-p.f)=,:
pushh: dits2L =na; b:
```


COMPARACIÓN: AFD vs AUTÓMATA DE PILA



CARACTERÍSTICA AFD AUTÓMATA DE PILA

Memoria Finita	Ilimitada
Tipo de memoria	Sin memoria Pila LIFO
Lenguajes Regulares	Libres de Contexto
reconocidos	(Tipo 3) (Tipo 2)
Capacidad de contar	No puede Sí puede
Estructuras anidadas	No puede Sí puede

JERARQUÍA DE CHOMSKY

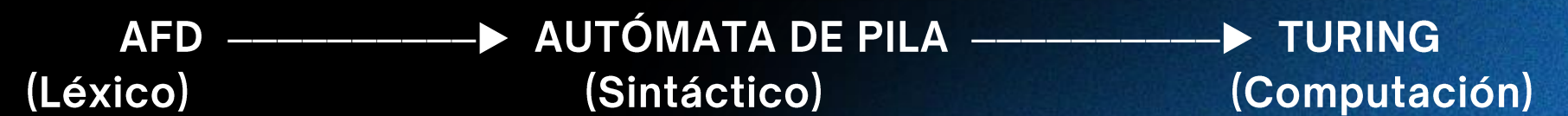
Máquinas de Turing (Tipo 0)
↑
Lenguajes Sensibles al Contexto (Tipo 1)
↑
► LENGUAJES LIBRES DE CONTEXTO (Tipo 2) ◀
↑
Lenguajes Regulares (Tipo 3)

CONCLUSION



El autómata de pila es el puente conceptual entre los autómatas finitos y las máquinas de Turing.

- Permite CONTAR y manejar estructuras ANIDADAS
- Base de los ANALIZADORES SINTÁCTICOS en compiladores (YACC, Bison)
- Limitación: Solo accede a la CIMA de la pila



Actividad 2:

Lexer para Docker con Expresiones Regulares

Sheen Albuquerque

- ```

 ecaier'pyhor=egulat((atatab-gylar-pyoc-batly-mec-olac-
 caniael_dre tokayar eminatorate))

 n-cwprikalz'()
 %%
 sta(compler({})
 smi7'e\ opse (hariant sative), saltarcamatiBoc.Bocallinge

 token, ragerf=tacompat,

 <Fgul eavE -8.=--of"P3tatty")
 evcra\ecurampert
 male\in motorome.
 da:ihltreage.({*nanmp/e.nrslit*)})\
))

 <F arrcwpelsalzE patl. To'lnise.Tryc(
 inated, iai-bey)
 in.Singac"Fght"erinlaelocage
 salle Cpv(one,
 sling. =r=
 (haqout, (not be))

```

# Construcción y Estructura del código

## 1. Definición de tokens (nombre, expresión\_regular)

## 2. Función lexer(texto)

- Recorre el texto buscando patrones
- Clasifica cada token
- Ignora espacios y comentarios

## 3. Función main()

- Lee el archivo
- Ejecuta el lexer
- Muestra resultados en tabla

```
docker_lexer.py > ...
1 import re
2
3 # Lista de tokens: (nombre_token, expresión_regular)
4 tokens = [
5 # Palabras clave de Dockerfile
6 ('FROM', r'\bFROM\b'),
7 ('RUN', r'\bRUN\b'),
8 ('COPY', r'\bCOPY\b'),
9 ('ADD', r'\bADD\b'),
10 ('WORKDIR', r'\bWORKDIR\b'),
11 ('ENV', r'\bENV\b'),
12 ('EXPOSE', r'\bEXPOSE\b'),
13 ('CMD', r'\bCMD\b'),
14 ('ENTRYPOINT', r'\bENTRYPOINT\b'),
15 ('ARG', r'\bARG\b'),
16 ('LABEL', r'\bLABEL\b'),
17 ('VOLUME', r'\bVOLUME\b'),
18 ('USER', r'\bUSER\b'),
19 ('ONBUILD', r'\bONBUILD\b'),
20 ('STOPSIGNAL', r'\bSTOPSIGNAL\b'),
21 ('HEALTHCHECK', r'\bHEALTHCHECK\b'),
22 ('SHELL', r'\bSHELL\b'),
23
24 # Identificadores (nombres de variables, rutas)
25 ('IDENTIFIER', r'[a-zA-Z0-9_][a-zA-Z0-9_./-]*'),
26
27 # Números
28 ('NUMBER', r'\b\d+(\.\d+)?\b'),
29
30 # Operadores y símbolos
31 ('EQUALS', r'='),
32 ('COMMA', r','),
33 ('LBRACKET', r'\['),
34 ('RBRACKET', r'\]'),
35 ('LBRACE', r'\{'),
36 ('RBRACE', r'\}'),
37 ('SEMICOLON', r';'),
38
39 # Caracteres especiales válidos en Dockerfile
40 ('COLON', r':'),
41 ('SLASH', r'/'),
42 ('DOT', r'\.'),
43 ('DASH', r'-'),
44
45 # Espacios y comentarios
46 ('SPACE', r'\s'),
47 ('COMMENT', r'#.*\n')
48]
49
50 def lexer(texto):
51 """Analiza el texto y genera tokens."""
52 patron = '|'.join(f'(?P<{nombre}>{regex})' for nombre, regex in tokens)
53 linea = 1
54 inicio_linea = 0
55
56 for coincidencia in re.finditer(patron, texto):
57 tipo = coincidencia.lastgroup
58 valor = coincidencia.group(tipo)
59
60 if tipo == 'NEWLINE':
61 inicio_linea = coincidencia.end()
62 linea += 1
63 elif tipo == 'SKIP' or tipo == 'COMMENT':
64 continue
65 elif tipo == 'ERROR':
66 columna = coincidencia.start() - inicio_linea + 1
67 yield (tipo, valor, linea, columna)
68 else:
69 columna = coincidencia.start() - inicio_linea + 1
70 yield (tipo, valor, linea, columna)
71
72 def main():
73 import sys
74 import os
75
76 # Si se pasa un argumento, usar ese archivo
77 if len(sys.argv) > 1:
78 nombre_archivo = sys.argv[1]
79 else:
80 # Si no, usar pruebas/Dockerfile1 por defecto
81 nombre_archivo = 'pruebas/Dockerfile1'
82
83 # Verificar si el archivo existe
84 if not os.path.exists(nombre_archivo):
85 print(f"❌ Error: El archivo '{nombre_archivo}' no fue encontrado.")
86 print("⚠️ Asegúrate de tener archivos en la carpeta 'pruebas'")
87 return
88
89 try:
90 with open(nombre_archivo, 'r', encoding='utf-8') as archivo:
91 texto = archivo.read()
92 tokens = lexer(texto)
93 # Mostrar tokens en una tabla (simulada)
94 # ... (código para mostrar la tabla) ...
95 except Exception as e:
96 print(f"Error al leer el archivo: {e}")
97
98 if __name__ == '__main__':
99 main()
```



# Ejemplo 1

pruebas > ≡ Dockerfile1

```
1 # Dockerfile1 - Imagen base simple
2 FROM ubuntu:22.04
3 RUN apt-get update
4 CMD ["echo", "Hola mundo"]
```



| TOKEN      | LEXEMA  | LÍNEA | COLUMNA |
|------------|---------|-------|---------|
| FROM       | FROM    | 2     | 1       |
| IDENTIFIER | ubuntu  | 2     | 6       |
| COLON      | :       | 2     | 12      |
| IDENTIFIER | 22.04   | 2     | 13      |
| RUN        | RUN     | 3     | 1       |
| IDENTIFIER | apt-get | 3     | 5       |
| IDENTIFIER | update  | 3     | 13      |
| CMD        | CMD     | 4     | 1       |
| LBRACKET   | [       | 4     | 5       |
| QUOTE      | "       | 4     | 6       |
| IDENTIFIER | echo    | 4     | 7       |
| QUOTE      | "       | 4     | 11      |
| COMMA      | ,       | 4     | 12      |
| QUOTE      | "       | 4     | 14      |
| IDENTIFIER | Hola    | 4     | 15      |
| IDENTIFIER | mundo   | 4     | 20      |
| QUOTE      | "       | 4     | 25      |
| RBRACKET   | ]       | 4     | 26      |

# Ejemplo 2

pruebas > ≡ Dockerfile2

```
1 # Dockerfile2 - Configuración de entorno
2 FROM python:3.12
3 WORKDIR /app
4 COPY . /app
5 ENV PORT=8080
6 EXPOSE 8080
7 CMD ["python", "app.py"]
```



| TOKEN      | LEXEMA  | LÍNEA | COLUMNA |
|------------|---------|-------|---------|
| FROM       | FROM    | 2     | 1       |
| IDENTIFIER | python  | 2     | 6       |
| COLON      | :       | 2     | 12      |
| IDENTIFIER | 3.12    | 2     | 13      |
| WORKDIR    | WORKDIR | 3     | 1       |
| SLASH      | /       | 3     | 9       |
| IDENTIFIER | app     | 3     | 10      |
| COPY       | COPY    | 4     | 1       |
| DOT        | .       | 4     | 6       |
| SLASH      | /       | 4     | 8       |
| IDENTIFIER | app     | 4     | 9       |
| ENV        | ENV     | 5     | 1       |
| IDENTIFIER | PORT    | 5     | 5       |
| EQUALS     | =       | 5     | 9       |
| IDENTIFIER | 8080    | 5     | 10      |
| EXPOSE     | EXPOSE  | 6     | 1       |
| IDENTIFIER | 8080    | 6     | 8       |
| CMD        | CMD     | 7     | 1       |
| LBRACKET   | [       | 7     | 5       |
| QUOTE      | "       | 7     | 6       |
| IDENTIFIER | python  | 7     | 7       |
| QUOTE      | "       | 7     | 13      |
| COMMA      | ,       | 7     | 14      |
| QUOTE      | "       | 7     | 16      |
| IDENTIFIER | app.py  | 7     | 17      |
| QUOTE      | "       | 7     | 23      |
| RBRACKET   | ]       | 7     | 24      |



# Ejemplo 3

```
pruebas > Dockerfile_error
```

```
1 FROM ubuntu
2 RUN apt-get update @
3 COPY . /app
```

| TOKEN      | LEXEMA  | LÍNEA | COLUMNA |
|------------|---------|-------|---------|
| FROM       | FROM    | 1     | 1       |
| IDENTIFIER | ubuntu  | 1     | 6       |
| RUN        | RUN     | 2     | 1       |
| IDENTIFIER | apt-get | 2     | 5       |
| IDENTIFIER | update  | 2     | 13      |
| AT         | @       | 2     | 20      |
| COPY       | COPY    | 3     | 1       |
| DOT        | .       | 3     | 6       |
| SLASH      | /       | 3     | 8       |
| IDENTIFIER | app     | 3     | 9       |



\_\_\_\_\_

- ```

cmnt Juneagpt.4);
cent -coll =tabrenplanes-97 "wrietypa
cmnt 1jamdo()/irl3.2 sotastop)
};
ccerge rintine=lvf.-duoluction; estimate;
cmntgot);
cameronf(; Vnsaue ereewtie(stion'a-K1444);
cdntestentsof));
ecipo ( armanict:
cmpt rsolee~salnee().tove"((veree)) "
cmntcarlgolo ooft_dunipagte,cef)) +
cmpt cōentll." etanot;
cmpt=onhi; of(
  ion( el, fcll,3ed)

```

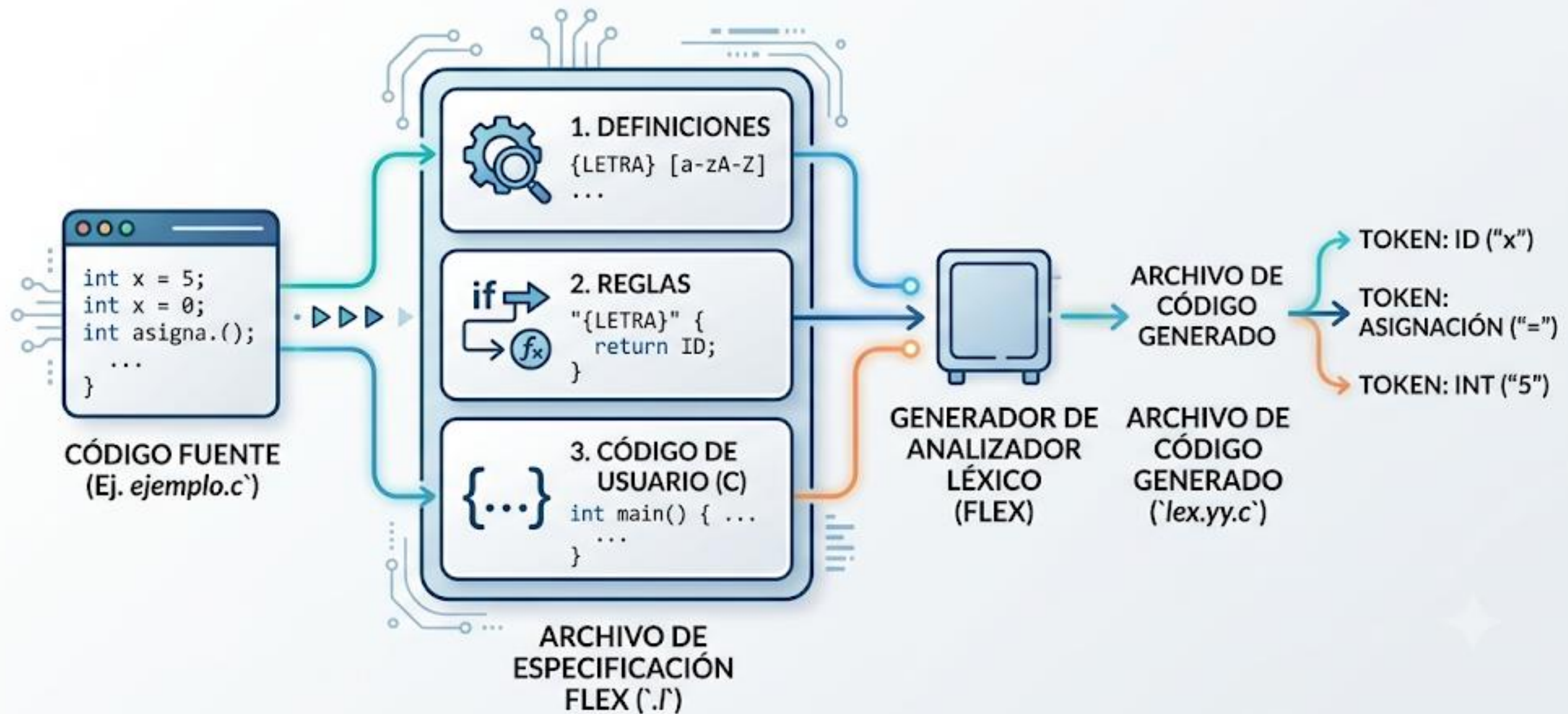
Actividad 3: Analizador Lexico usando metacompilador Flex

Lenguaje L subconjunto de Rust

Carlos David Garcia

```
13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000
```


ANÁLISIS LÉXICO Y FLEX



DEFINICIÓN DEL LENGUAJE L

Token	Descripción	Patrón (e
KEYWORD	Palabras reservadas	fn, let, mu struct, en const
TYPE	Tipos primitivos	i8, i16, i32 String, Ve
BOOL	Literales booleanos	true fals
INT_SUFFIX	Entero con sufijo de tipo	[0-9]+(i8\
FLOAT	Número flotante	[0-9]+\.[0-
INT	Entero decimal simple	[0-9]+
CHAR	Literal de carácter (con	'([^\\"\\n]\
STRING	Literal de cadena (con	"([^\\"\\n\
IDENTIFIER	Identificador (no palabra	[a-zA-Z_]([
OP	Operadores de un carácter	+, -, *, /, %
LPAREN	Paréntesis izquierdo	(
RPAREN	Paréntesis derecho	)
LBRACE	Llave izquierda	{
RBRACE	Llave derecha	}
(ignorado)	Comentario de línea	//.*
(ignorado)	Comentario de bloque	/*...*/ (es
(ignorado)	Espacios en blanco	[\t\r\n]+
ERROR	Carácter no reconocido	. (cualqui

Conclusion

- Flex automatiza la construcción del lexer, reduciendo errores y tiempo.
- La definición clara del lenguaje L y el orden de reglas son clave.
- La herramienta es extensible a más tokens del lenguaje Rust.
- Implementacion de otras herramientas como Bison

```
501 // abocattic, livax; (bel());  
5521 // fe; /sennantur, let,  
511 // tut; flexis=retogardh );  
521 // mstela Pain/Ssion; h(ol faw))  
511 // nspmatiaab. a'sBocab wilet3vf),  
511 // abocattic, livax; (bel());
```


Actividad 4:
FLEX en Seguridad Informática

Por: Génesis Varguillas

Un puente entre el rendimiento nativo y la versatilidad moderna

FLEX no trabaja solo. Genera código nativo altamente optimizado en C o C++.



En el ecosistema actual, los ingenieros compilan el analizador de FLEX como una biblioteca compartida controlada por Python. Esto permite buscar amenazas con la agilidad de los scripts modernos, pero ejecutando la tokenización a la velocidad implacable de C.

Tres frentes de batalla simultáneos



Servidores (Análisis de Logs)

Procesamiento de millones de líneas en Apache o Nginx para detectar **Inyecciones SQL** o **ataques de fuerza bruta** en milisegundos.



Redes (Análisis de Tráfico)

Extracción quirúrgica y en tiempo real de **información** específica oculta dentro de los **flujos de protocolos** de red.



Archivos (Análisis de Malware)

Diseccción rápida de **scripts maliciosos**, inspeccionando código **ofuscado** en **JavaScript** o **VBScript** para neutralizar amenazas.

El ADN compartido por los titanes de la industria

Las herramientas que los profesionales de la seguridad usan a diario están construidas sobre arquitecturas léxicas idénticas.



Radiografía léxica: Una regla de Snort

```
alert tcp 192.168.1.1 any -> any 80 (msg:"Intrusión  
Web Detectada"; content:"malware"; sid:100001;)
```

```
graph TD; Rule[alert tcp 192.168.1.1 any -> any 80 (msg: \"Intrusión Web Detectada\"; content: \"malware\"; sid: 100001;)] --- Actions[Acciones de Seguridad<br/>Tokeniza alert, log, pass.]; Rule --- Protocols[Protocolos<br/>Mapeo exacto para tcp, udp, icmp.]; Rule --- Directions[Direcciones e IPs<br/>Usa regex para capturar estructuras separadas por puntos y variables como HOME_NET.]; Rule --- Flow[Flujo y Opciones<br/>Identifica operadores de dirección y opciones internas de firma.];
```

Acciones de Seguridad

Tokeniza alert, log, pass.

Protocolos

Mapeo exacto para tcp, udp, icmp.

Direcciones e IPs

Usa regex para capturar estructuras separadas por puntos y variables como HOME_NET.

Flujo y Opciones

Identifica operadores de dirección y opciones internas de firma.

El archivo **snort_lexer.l** utiliza la función **yylex()** para evaluar esta declaración y alertar la intrusión sin degradar la velocidad de la red.

Clasificando Malware con YARA

YARA es el estándar para buscar patrones en archivos sospechosos. Su núcleo utiliza rigurosamente FLEX para clasificar la anatomía del malware.

```
0000: 4D 5A 90 00 03 00 00 00 04 00 00 00 .....
0001: FF FF 00 00 B8 00 00 00 00 00 00 00 .....
0002: 40 00 00 00 00 00 00 00 00 00 00 00 .....
0003: 00 00 00 00 00 00 00 00 00 00 00 00 .....
0004: 00 00 00 00 00 00 00 00 00 00 00 00 .....
0004: 00 00 00 00 00 00 00 00 00 00 00 00 .....
0005: 00 00 00 00 00 00 00 00 00 00 00 00 .....
0006: 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007: D8 00 00 00 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C
0068: CD 21 54 68 69 09 73 20 70 72 6F 67 67 72 61 6D
006E: 20 63 61 61 6E 6E 6F 74 20 62 65 20 20 72 75 6E
007A: 20 69 6E 20 44 4F 53 20 6D 6F 64 65 2E 0D 0D 0A
007A: 24 00 00 00 00 00 00 ...
```

Palabras Clave

Identifica rule, meta, strings, condition y genera tokens como TOKEN_RULE.

Identificadores Especiales

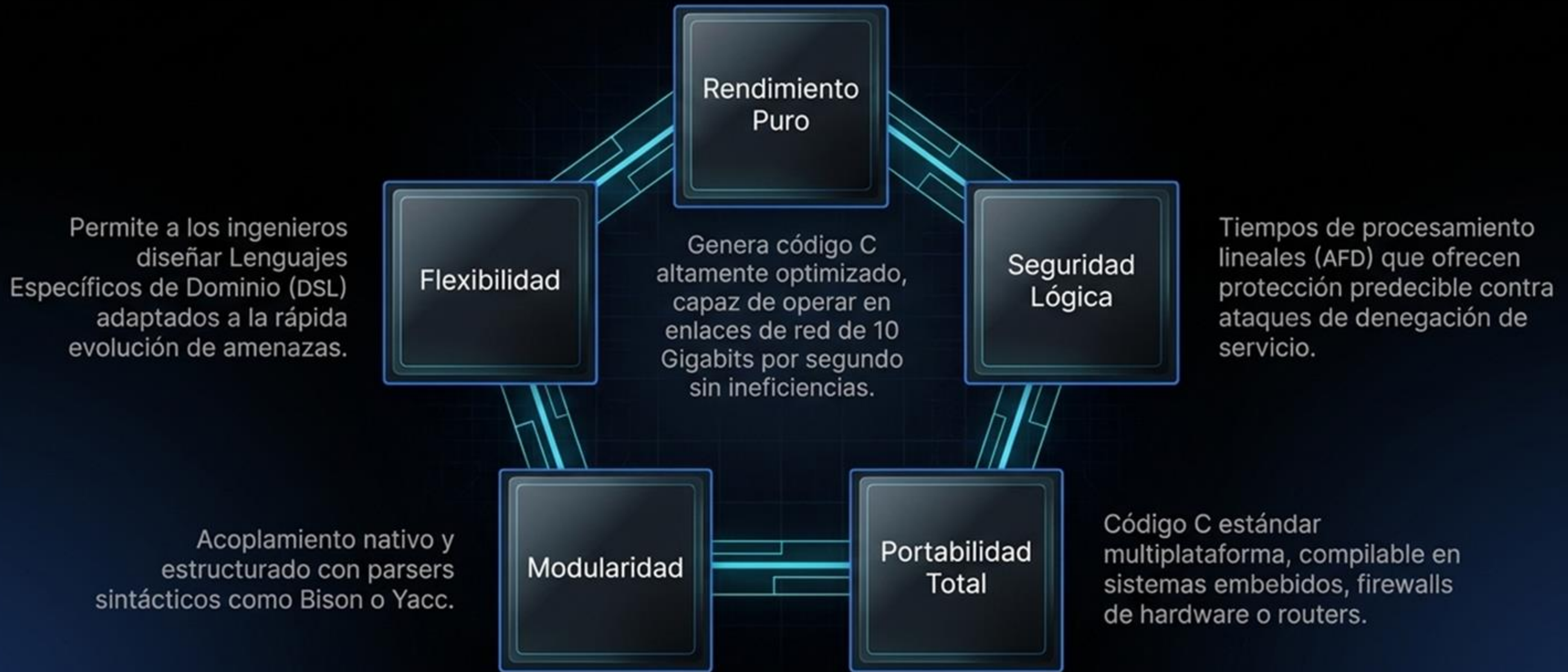
Mapea prefijos vitales: \$ (Identificador de cadena), # (Conteo de ocurrencias), @ (Offsets en memoria).

Tipos de Cadenas

El lexer discrimina entre texto tradicional, patrones hexadecimales entre llaves { }, y expresiones regulares.

FLEX procesa la regla de forma lineal y exacta, devolviendo valores numéricos enteros para cada lexema encontrado en memoria.

La arquitectura de la defensa moderna: 5 Pilares



**¡MUCHAS
GRACIAS!**